

ODI Demo Build — 3-Session Plan (3 x 1 hour)

Cold start, no prework. Boss hands-on, you drive the risky parts. Each session is self-contained and resumable.

Session 1 opens with a 5-minute access check (these aren't set up yet)

Do this first, or the AWS build can't start. Tools (terraform, aws, jq, node, dbt) are already installed; access is not.

- **Fivetran:** log in, generate an API key, then in the terminal export FIVETRAN_API_KEY=key:secret
- **AWS:** run aws configure (paste an admin access key + secret, or use SSO), confirm with aws sts get-caller-identity
- **If either isn't available:** don't burn the hour fighting it. Pivot to touring a finished demo (dozens in ~/Documents/GitHub/*-ODI-Demo) and locking the vertical/persona, then provision next session once access is sorted.

Carry between sessions (no prework assumed — re-establish at the top of each)

- The **Fivetran API key** — the env var won't survive a new terminal, so re-export it each session (store it somewhere safe).
- The **demo directory path** Claude created, and your **AWS creds** (re-run aws configure if needed).
- The **group / connector / destination ids** — Claude can re-list these from the API if you lose them.

Session 1 (today) — Foundation & first data

Goal: AWS lake provisioned, MDLS destination + OAuth connector created, first sync started. Lock the vertical, org name, and persona before you provision.

Kickoff

Let's build a new ODI demo with the odi-demo-builder skill. Vertical: [VERTICAL]. Org name: [ORG]. Buyer persona: [PERSONA]. Nothing is provisioned yet — my AWS creds and \$FIVETRAN_API_KEY are now set. Scaffold the Terraform for the S3 bucket, Glue bronze/silver/gold, and the Fivetran IAM role, and walk me through provisioning step by step.

Provision the lake

Start the MDLS destination setup so we get the AWS account id and external id, put them into tfvars, then run terraform init and apply. Show me the bucket name and IAM role ARN outputs.

This is the handshake: you begin the destination in the Fivetran UI to get the external id, paste it into tfvars, then apply the Terraform. Claude does the rest.

Create the destination

Create the Managed Data Lake destination via the REST API using the terraform outputs (bucket, IAM role ARN, Glue bronze database) in us-east-1. Confirm the destination id and setup status.

OAuth connector

Create the [source] connector in this group via the REST API, then generate a Fivetran Connect Card link and give me the URL so I can sign in with OAuth.

In the browser: open the Connect Card URL, sign in via OAuth, pick the data, click Save. No passwords typed.

Test + sync

Run the connector setup test (expect CONNECTED), then trigger the first sync and poll sync_state every 30 seconds until it's SYNCED. Tell me which tables landed.

Session 1 done

Lake provisioned + connector authorized + first sync landing in the Iceberg lake. If AWS setup ran long, stopping after the destination/connector are created and the sync is triggered is a fine session 1 — verify and pick up at dbt next time.

Session 2 — dbt pipeline (bronze / silver / gold)

Resume check: re-export FIVETRAN_API_KEY, confirm aws sts get-caller-identity works, and confirm session 1's sync finished.

Resume

Resume the ODI demo in [demo dir]. Verify the Fivetran sync completed and list the tables that landed in the Glue bronze database.

Scaffold dbt

Scaffold the dbt project to the portfolio standard: bronze sources.yml from the synced tables, silver staging models, and one gold Iceberg fact/dimension. Apply Niraj's content rules — org name [ORG] everywhere, every connector gets its Fivetran deep link, no raw table names exposed.

dbt build

Run dbt build --select bronze silver gold and confirm the gold Iceberg tables are created in the Glue catalog. If it fails, diagnose from the error before fixing.

Verify

Show me a sample query against the gold tables through Athena so we can see the modeled data.

Session 2 done

Working bronze/silver/gold pipeline, gold Iceberg tables queryable in the lake.

Session 3 — Frontend, polish & demo tour

Resume check: same env re-establish as session 2.

Frontend shell

Generate the React frontend shell — a Landing page with the [ORG] hero and a pipeline diagram (Fivetran -> S3/Iceberg -> dbt -> Athena), and a Pipeline page listing the connector with its Fivetran deep link.

Design pass

Run the frontend-design pass for the [VERTICAL] vertical and [PERSONA] buyer — distinctive look, no generic AI aesthetic, analytical charts instead of dense tables, no emojis.

Content QA

Do a Niraj content QA pass: org name [ORG] in every title, H1, and navbar; no raw table names exposed; every connector shows its Fivetran deep link.

Tour

Start the frontend and walk me through the demo end to end — landing, pipeline, the data in the lake.

Session 3 done

Demo-ready ODI app: live connector, Iceberg lake, dbt pipeline, and a polished frontend you can present.